

FR. CONCEICAO RODRIGUES COLLEGE OF ENGINEERING

Department of Computer Engineering Academic Year 2025-26

Rubrics for Lab Experiments

**Class : B.E. Computer
Engineering**

Subject Name :DC Lab

Semester : VIII

Subject Code : CSL801

Practical No:	6
Title:	To implement Clock Synchronization/ Mutual Exclusion algorithms.
Date of Performance:	15/11/25
Roll No:	9913
Name of the Student:	Mark Lopes

Evaluation:

Performance Indicator	Below average	Average	Good	Excellent	Marks
On time Submission (2)	Not submitted (0)	Submitted after deadline (1)	Early or on time submission(2)	---	
Test cases and output (4)	Incorrect output (1)	The expected output is verified only a for few test cases (2)	The expected output is Verified for all test cases but is not presentable (3)	Expected output is obtained for all test cases. Presentable and easy to follow (4)	
Coding efficiency (2)	The code is not structured at all (0)	The code is structured but not efficient (1)	The code is Structured and efficient. (2)	-	
Knowledge(2)	Basic concepts not clear (0)	Understood the basic concepts (1)	Could explain the concept with suitable example (1.5)	Could relate the theory with real world application(2)	
Total					

LAB 6

Aim: To implement Clock Synchronization/ Mutual Exclusion algorithms.

Theory:

Clock Synchronization:

The Lamport algorithm is a logical clock synchronization algorithm used in distributed systems to establish a partial ordering of events. The algorithm is based on the concept of logical clocks, which assign a timestamp to each event in a distributed system.

In Lamport's algorithm, each process maintains a logical clock that ticks whenever an event occurs at that process. The logical clock value assigned to an event is the maximum of the current logical clock value of the process and the timestamp of the incoming message that triggered the event.

The logical clocks in Lamport's algorithm satisfy the following two properties:

- If event A happens before event B, then the logical clock value of A is less than the logical clock value of B.
- If events A and B are concurrent, then the logical clock value of A is not equal to the logical clock value of B.

Using Lamport's algorithm, processes can synchronize their logical clocks so that they agree on the ordering of events. This enables distributed systems to reason about causality, and to detect and resolve conflicts that may arise due to concurrent events.

Lamport Algorithm

Lamport developed a "happens-before" notation to express this: If a and b are events in the same process, and a occurs before b, then $a \rightarrow b$ is true. If a is the event of a message being sent by one process, and b is the event of the message being received by another process, then $a \rightarrow b$. This relationship is transitive i.e. $a \rightarrow b$ and $b \rightarrow c$ then $a \rightarrow c$. Satisfying conditions for implementing clock: If $a \rightarrow b$ then $c(a) < c(b)$. Implementation of logical clock:

Condition 1: If a and b are two events within the same process P_i and a occur before b then $C_i(a) < C_i(b)$.

Condition 2: if a is the sending of a message by process P_i and b is the receipt of that message by process P_j then $C_i(a) < C_j(b)$.

Condition 3: A clock C_i associated with a process P_i must always go forward never backward that is correction to the time of clock is done by +ive adding.

Total ordering and Partial ordering

Total ordering is an ordering that defines the exact order of every element in the series.

Partial ordering of elements in a series is an ordering that doesn't specify the exact order of every item, but only defines the order between certain key items that depend on each other.

The meaning of these words is the same in the context of distributed computing. The only significance of distributed computing to these terms is the fact that partial ordering of events is much commoner than total ordering. In a local, single-threaded application, the order in which events happen is ordered, implicitly, since the CPU can only do one thing at a time. In a distributed system, you generally only coordinate a partial ordering of those events that have a dependency on one another and let other events happen in whatever order they happen.

Example, taken from the comments: If you have three events $\{A, B, C\}$, then they are ordered if they always have to happen in the order $A > B > C$. However, if A must happen before C, but you don't care when B happens, then they are partially ordered. In this case we would say that the sequences $A > B > C$, $A > C > B$, and $B > A > C$ all satisfy the partial ordering.

Mutual Exclusion in Distributed System:

Mutual exclusion is a concurrency control property which is introduced to prevent race conditions. It is the requirement that a process cannot enter its critical section while another concurrent process is currently present or executing in its critical section i.e., only one process is allowed to execute the critical section at any given instance of time.

In Distributed systems, we neither have shared memory nor a common physical clock and therefore we cannot solve mutual exclusion problem using shared variables. To eliminate the mutual exclusion problem in distributed system approach based on message passing is used.

Requirements of Mutual exclusion Algorithm:

- No Deadlock:

Two or more site should not endlessly wait for any message that will never arrive.

- No Starvation:

Any site should not wait indefinitely to execute critical section while other sites are repeatedly executing critical section

- Fairness:

Each site should get a fair chance to execute critical section. Any request to execute critical section must be executed in the order they are made.

- Fault Tolerance:

In case of failure, it should be able to recognize it by itself in order to continue functioning without any disruption.

Solution to distributed mutual exclusion:

As we know shared variables or a local kernel cannot be used to implement mutual exclusion in distributed systems. Message passing is a way to implement mutual exclusion. Below are

the three approaches based on message passing to implement mutual exclusion in distributed systems:

Token Based Algorithm:

- A unique token is shared among all the sites.
- If a site possesses the unique token, it is allowed to enter its critical section
- This approach uses sequence number to order requests for the critical section.
- Each requests for critical section contains a sequence number. This sequence number is used to distinguish old and current requests.
- This approach insures Mutual exclusion as the token is unique
- Example: Suzuki-Kasami's Broadcast Algorithm

Non-token based approach:

- A site communicates with other sites in order to determine which sites should execute critical section next. This requires exchange of two or more successive round of messages among sites.
- This approach use timestamps instead of sequence number to order requests for the critical section.
- When ever a site make request for critical section, it gets a timestamp. Timestamp is also used to resolve any conflict between critical section requests.
- All algorithm which follows non-token based approach maintains a logical clock. Logical clocks get updated according to Lamport's scheme
- Example: Lamport's algorithm, Ricart-Agrawala algorithm

Quorum based approach:

- Instead of requesting permission to execute the critical section from all other sites, Each site requests only a subset of sites which is called a quorum.
- Any two subsets of sites or Quorum contains a common site.
- This common site is responsible to ensure mutual exclusion
- Example: Maekawa's Algorithm

Implementation:

Suzuki-Kasami Algorithm:

1. To enter Critical section:

- When a site S_i wants to enter the critical section and it does not have the token then it increments its sequence number $RNi[i]$ and sends a request message $REQUEST(i, sn)$ to all other sites in order to request the token.

Here sn is update value of $RNi[i]$

- When a site S_j receives the request message $REQUEST(i, sn)$ from site S_i , it sets $RNj[i]$ to maximum of $RNj[i]$ and sn i.e $RNj[i] = \max(RNj[i], sn)$.
- After updating $RNj[i]$, Site S_j sends the token to site S_i if it has token and $RNj[i] = LN[i] + 1$

2. To execute the critical section:

- Site S_i executes the critical section if it has acquired the token.

3. To release the critical section:

After finishing the execution Site S_i exits the critical section and does following:

- sets $LN[i] = RNi[i]$ to indicate that its critical section request $RNi[i]$ has been executed
- For every site S_j , whose ID is not present in the token queue Q , it appends its ID to Q if $RNi[j] = LN[j] + 1$ to indicate that site S_j has an outstanding request.

- After above updation, if the Queue Q is non-empty, it pops a site ID from the Q and sends the token to site indicated by popped ID.
- If the queue Q is empty, it keeps the token

Message Complexity:

The algorithm requires 0 message invocation if the site already holds the idle token at the time of critical section request or maximum of N message per critical section execution.

This N messages involves

- (N – 1) request messages
- 1 reply message

(Code & Output:)

SuzukiKasamiClockSync.java

```
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;

public class SuzukiKasamiClockSync {
    public static void main(String[] args) throws Exception {
        final int N = 5; // number of sites
        final int simulationSeconds = 20;

        // shared message queues (one per node)
        List<BlockingQueue<Message>> queues = new ArrayList<>();
        for (int i = 0; i < N; i++)
            queues.add(new LinkedBlockingQueue<>());

        // create a time server
        TimeServer timeServer = new TimeServer();
        new Thread(timeServer, "TimeServer").start();

        // initial token is given to node 0
        Token initialToken = new Token(N);
        initialToken.LN[0] = 0;

        // create and start nodes
        List<Node> nodes = new ArrayList<>();
        for (int i = 0; i < N; i++) {
            boolean hasToken = (i == 0);
            Node node = new Node(i, N, queues, hasToken ? initialToken : null, timeServer);
            nodes.add(node);
            new Thread(node, "Node-" + i).start();
        }
    }
}
```

```

        // let simulation run
        Thread.sleep(simulationSeconds * 1000L);

        // stop nodes
        for (Node n : nodes) n.shutdown();
        timeServer.shutdown();

        System.out.println("Simulation finished.");
    }
}

// =====
// Message Class
// =====

class Message {
    enum Type { REQUEST, TOKEN }

    final Type type;
    final int senderId;
    final int sn;
    final Token token;
    final int lamport;

    Message(Type type, int senderId, int sn, Token token, int lamport) {
        this.type = type;
        this.senderId = senderId;
        this.sn = sn;
        this.token = token;
        this.lamport = lamport;
    }

    static Message request(int from, int sn, int lamport) {
        return new Message(Type.REQUEST, from, sn, null, lamport);
    }

    static Message token(int from, Token token, int lamport) {
        return new Message(Type.TOKEN, from, -1, token, lamport);
    }
}

// =====
// Token Class
// =====

```

```

class Token {
    int[] LN;
    Deque<Integer> Q;

    Token(int n) {
        LN = new int[n];
        Q = new ArrayDeque<>();
        Arrays.fill(LN, 0);
    }

    Token deepCopy() {
        Token t = new Token(LN.length);
        System.arraycopy(this.LN, 0, t.LN, 0, LN.length);
        t.Q.addAll(this.Q);
        return t;
    }
}

// =====
// Time Server (Cristian Algorithm)
// =====

class TimeServer implements Runnable {
    private volatile boolean running = true;

    long now() { return System.currentTimeMillis(); }

    public void shutdown() { running = false; }

    public void run() {
        while (running) {
            try { Thread.sleep(1000); }
            catch (InterruptedException e) { break; }
        }
    }
}

// =====
// Node Implementation (Suzuki–Kasami + Lamport + Cristian)
// =====

class Node implements Runnable {
    final int id, N;
    final List<BlockingQueue<Message>> queues;
    final BlockingQueue<Message> myQueue;

```

```

final TimeServer timeServer;

volatile boolean running = true;

int[] RN;
Token token;
boolean hasToken = false;

final AtomicInteger mySeq = new AtomicInteger(0);
int lamport = 0;

final Random rnd = new Random();

Node(int id, int N, List<BlockingQueue<Message>> queues, Token initialToken, TimeServer timeServer) {
    this.id = id;
    this.N = N;
    this.queues = queues;
    this.myQueue = queues.get(id);
    this.timeServer = timeServer;

    this.RN = new int[N];
    Arrays.fill(RN, 0);

    if (initialToken != null) {
        this.token = initialToken;
        this.hasToken = true;
    }
}

// -----
// Request Critical Section
// -----
void requestCS() {
    lamport++;

    int sn = mySeq.incrementAndGet();
    RN[id] = sn;

    System.out.println(timestamp() + "Node " + id + " requests CS with sn=" + sn +
        " (lamport=" + lamport + ")");

    if (!hasToken) {
        for (int j = 0; j < N; j++) {
            if (j == id) continue;
            sendMessage(j, Message.request(id, sn, lamport));
        }
    }
}

```



```

    }
}

// wait until token arrives
while (running && !hasToken) {
    try { Thread.sleep(20); }
    catch (InterruptedException e) { break; }
}

if (hasToken) {
    enterCS();
    releaseCS();
}
}

void enterCS() {
    lamport++;
    System.out.println(timestamp() + "**** Node " + id + " ENTERING CS (lamport=" +
        lamport + ") ****");

    try { Thread.sleep(200 + rnd.nextInt(300)); }
    catch (InterruptedException ignored) {}

    System.out.println(timestamp() + "**** Node " + id + " EXITING CS ****");
}

// -----
// Release Critical Section
// -----
void releaseCS() {
    for (int j = 0; j < N; j++) {
        if (j == id) continue;

        boolean inQueue = token.Q.contains(j);

        if (!inQueue && RN[j] == token.LN[j] + 1)
            token.Q.addLast(j);
    }

    if (!token.Q.isEmpty()) {
        int dest = token.Q.pollFirst();
        Token sendTok = token.deepCopy();
        hasToken = false;
        token = null;
    }
}

```

```

        System.out.println(timestamp() +
            "Node " + id + " sends token to " + dest);

        sendMessage(dest, Message.token(id, sendTok, lamport));
    } else {
        System.out.println(timestamp() +
            "Node " + id + " keeps token (Q empty)");
    }
}

// -----
// Handle Incoming Messages
// -----
void handleRequest(Message m) {
    int i = m.senderId;
    RN[i] = Math.max(RN[i], m.sn);
    lamport = Math.max(lamport, m.lamport) + 1;

    if (hasToken && RN[i] == token.LN[i] + 1) {
        Token sendTok = token.deepCopy();
        hasToken = false;
        token = null;

        System.out.println(timestamp() +
            "Node " + id + " forwards token to " + i);

        sendMessage(i, Message.token(id, sendTok, lamport));
    }
}

void handleToken(Message m) {
    this.token = m.token;
    this.hasToken = true;

    lamport = Math.max(lamport, m.lamport) + 1;

    System.out.println(timestamp() +
        "Node " + id + " received token (lamport=" + lamport + ")");
}

// -----
// Clock Sync: Cristian Algorithm
// -----
void syncClockWithServer() {
    long t0 = System.currentTimeMillis();

```

```

    long serverTime = timeServer.now();
    long t1 = System.currentTimeMillis();

    long rtt = t1 - t0;
    long estimated = serverTime + rtt / 2;

    System.out.println(timestamp() +
        "Node " + id + " sync: local=" + System.currentTimeMillis() +
        " server=" + serverTime + " rtt=" + rtt +
        " estimated=" + estimated);
}

// -----
// Send Message (with delay simulation)
// -----
void sendMessage(int to, Message m) {
    new Thread(() -> {
        try { Thread.sleep(10 + rnd.nextInt(50)); }
        catch (InterruptedException ignored) {}

        try { queues.get(to).put(m); }
        catch (InterruptedException ignored) {}
    }).start();
}

String timestamp() {
    return "[t=" + System.currentTimeMillis() + "] ";
}

// -----
// Node Main Loop
// -----
public void run() {
    // message processing thread
    Thread msgProc = new Thread(() -> {
        while (running) {
            try {
                Message m = myQueue.poll(200, TimeUnit.MILLISECONDS);
                if (m == null) continue;

                switch (m.type) {
                    case REQUEST: handleRequest(m); break;
                    case TOKEN:  handleToken(m); break;
                }
            }
            catch (InterruptedException e) { break; }
        }
    });
}

```

```

    }
});

msgProc.start();

// main behavior
try {
    while (running) {
        if (rnd.nextDouble() < 0.15) requestCS();
        if (rnd.nextDouble() < 0.07) syncClockWithServer();

        Thread.sleep(200 + rnd.nextInt(300));
    }
} catch (InterruptedException ignored) {}

msgProc.interrupt();
}

public void shutdown() { running = false; }
}

```

Welcome to Hallownest, little ghost...

Hallownest: C:\Users\Mark Lopes\Desktop\college\Sem_8\DC\exp6

● > javac .\SuzukiKasamiClockSync.java

Hallownest: C:\Users\Mark Lopes\Desktop\college\Sem_8\DC\exp6

● > java SuzukiKasamiClockSync

```
[t=1763994324619] Node 0 requests CS with sn=1 (lamport=1)
[t=1763994324620] *** Node 0 ENTERING CS (lamport=2) ***
[t=1763994324654] Node 4 requests CS with sn=1 (lamport=1)
[t=1763994324708] Node 0 forwards token to 4
[t=1763994324734] Node 1 requests CS with sn=1 (lamport=3)
[t=1763994324743] Node 4 received token (lamport=4)
[t=1763994324761] *** Node 4 ENTERING CS (lamport=5) ***
[t=1763994324776] Node 4 forwards token to 1
[t=1763994324781] Node 3 sync: local=1763994324781 server=1763994324781 rtt=0 estimated=1763994324781
[t=1763994324832] Node 1 received token (lamport=7)
[t=1763994324836] *** Node 1 ENTERING CS (lamport=8) ***
[t=1763994325104] *** Node 1 EXITING CS ***
[t=1763994325104] Node 1 sends token to 4
[t=1763994325106] *** Node 4 EXITING CS ***
Exception in thread "Node-4" java.lang.NullPointerException
    at Node.releaseCS(SuzukiKasamiClockSync.java:200)
    at Node.requestCS(SuzukiKasamiClockSync.java:178)
    at Node.run(SuzukiKasamiClockSync.java:310)
    at java.lang.Thread.run(Thread.java:750)
[t=1763994325113] *** Node 0 EXITING CS ***
Exception in thread "Node-0" java.lang.NullPointerException
    at Node.releaseCS(SuzukiKasamiClockSync.java:200)
    at Node.requestCS(SuzukiKasamiClockSync.java:178)
    at Node.run(SuzukiKasamiClockSync.java:310)
    at java.lang.Thread.run(Thread.java:750)
[t=1763994325136] Node 4 received token (lamport=9)
[t=1763994326003] Node 3 sync: local=1763994326003 server=1763994326003 rtt=0 estimated=1763994326003
[t=1763994326278] Node 2 requests CS with sn=1 (lamport=5)
[t=1763994326301] Node 4 forwards token to 2
[t=1763994326354] Node 2 received token (lamport=11)
[t=1763994326362] *** Node 2 ENTERING CS (lamport=12) ***
[t=1763994326691] *** Node 2 EXITING CS ***
[t=1763994326691] Node 2 sends token to 1
[t=1763994326691] Node 2 sync: local=1763994326691 server=1763994326691 rtt=0 estimated=1763994326691
[t=1763994326733] Node 1 received token (lamport=13)
[t=1763994326902] Node 2 requests CS with sn=2 (lamport=13)
[t=1763994329214] Node 1 requests CS with sn=2 (lamport=15)
[t=1763994329214] *** Node 1 ENTERING CS (lamport=16) ***
```

```

[t=1763994329214] *** Node 1 ENTERING CS (lamport=16) ***
[t=1763994329566] *** Node 1 EXITING CS ***
[t=1763994329566] Node 1 sends token to 4
[t=1763994329605] Node 4 received token (lamport=17)
[t=1763994330594] Node 3 requests CS with sn=1 (lamport=15)
[t=1763994330649] Node 4 forwards token to 3
[t=1763994330666] Node 3 received token (lamport=19)
[t=1763994330677] *** Node 3 ENTERING CS (lamport=20) ***
[t=1763994330964] *** Node 3 EXITING CS ***
[t=1763994330964] Node 3 sends token to 1
[t=1763994330966] Node 1 sync: local=1763994330966 server=1763994330966 rtt=0 estimated=1763994330966
[t=1763994330996] Node 1 received token (lamport=21)
[t=1763994331302] Node 1 requests CS with sn=3 (lamport=22)
[t=1763994331303] *** Node 1 ENTERING CS (lamport=23) ***
[t=1763994331528] *** Node 1 EXITING CS ***
[t=1763994331529] Node 1 sends token to 4
[t=1763994331561] Node 4 received token (lamport=24)
[t=1763994333561] Node 3 requests CS with sn=2 (lamport=21)
[t=1763994340132] Node 1 requests CS with sn=4 (lamport=25)
Simulation finished.

Halloween: C:\Users\Mark Lopes\Desktop\college\Sem_8\DC\exp6
>

```

Conclusion:

In summary, clock synchronization and mutual exclusion are essential to keep distributed systems consistent and coordinated. Physical clocks maintain real-world time, while logical clocks maintain event order. Token-based and non-token-based algorithms each solve mutual exclusion differently, with their own strengths and limitations. Together, these concepts ensure that distributed processes can operate safely, avoid conflicts, and maintain a reliable global order even without a single shared clock or memory.

Postlab Questions:

1. Distinguish between physical clock and logical clock synchronization

Physical clock synchronization tries to keep the real-time clocks of different machines consistent with actual time (UTC). It deals with clock drift, network delays, and hardware differences. Logical clock synchronization does not use real time; instead, it orders events based on causality. Logical clocks ensure happened-before ordering using counters or vectors, without referring to wall-clock time.

2. Show the calculation of the time interval between two synchronizations of a physical clock

Let ρ be the maximum drift rate of a clock and δ be the maximum allowed time error between two clocks. The worst-case drift between two clocks is $2\rho T$, because clocks may drift in opposite directions. To keep error within δ :

$$2\rho T \leq \delta$$

$$T \leq \delta / (2\rho)$$

Example: If $\rho = 10^{-5}$ and $\delta = 0.05$ seconds, then $T \leq 0.05 / (2 \times 10^{-5}) = 1250$ seconds.

3. How will you implement Logical clocks by using counters?

Logical clocks (Lamport clocks) are implemented with an integer counter at each process.

Before each local event: $LC = LC + 1$

Before sending a message: increment LC and include LC in the message.

On receiving a message with timestamp TS: set $LC = \max(LC, TS) + 1$

This ensures a consistent ordering of events across processes.

4. Give an example of partial and total ordering of events

Partial ordering example:

In process P1: $a \rightarrow b$

In process P2: $c \rightarrow d$

If a sends a message to c, then $a \rightarrow c$. But b and c are not comparable; they are concurrent.

This forms a partial order.

Total ordering example:

Assign timestamps as (LamportTime, ProcessID).

If two events have equal Lamport times, break ties using process ID.

For example: $(5,1) < (5,2)$. All events become comparable, forming a total order.

5. Compare Token based and Non-Token based algorithms in Distributed System.

Token-based algorithms use a unique token; only the token holder can enter the critical section. Examples include Suzuki–Kasami and Token Ring. Pros: no starvation, fewer messages when token is held. Cons: token loss halts the system and requires recovery.

Non-token-based algorithms use message exchanges to request and grant permission.

Examples: Lamport's mutual exclusion and Ricart–Agrawala. Pros: no dependence on a single token. Cons: higher message overhead and possible starvation if not controlled.